

Day 8 Learning (19/03/2026)

#Java Object Class: The Root of All Classes

The `java.lang.Object` class is the parent class of all classes in Java by default. It provides the fundamental behaviors that every Java object must possess.

1. Hierarchy and Role

- **Top Level:** Every class in Java, whether built-in or user-defined, implicitly extends the `Object` class.
- **Purpose:** It acts as a root to provide common methods like equality check, string representation, and thread synchronization.
- **Inheritance:** You don't need to write `extends Object`. The Java compiler adds it automatically.

2. Core Methods of Object Class

Method	Description
<code>toString()</code>	Returns a string representation of the object (Default: <code>ClassName@HashCode</code>).
<code>equals(Object obj)</code>	Indicates whether some other object is "equal to" this one (Default: Reference/Address comparison).
<code>hashCode()</code>	Returns a hash code value (integer) for the object, used in hashing-based collections.
<code>getClass()</code>	Returns the runtime class of the object. It is a final method.
<code>clone()</code>	Creates and returns a copy of this object. Requires the <code>Cloneable</code> interface.

wait/notify/notifyAll	Used for inter-thread communication in Multi-threading.
------------------------------	---

#Internal Working of HashMap and HashSet

`HashMap` and `HashSet` rely heavily on the `hashCode()` and `equals()` methods of the `Object` class to store and retrieve data efficiently.

1. The Data Structure (Buckets)

Internally, `HashMap` uses an **Array of Nodes** (also called Buckets). Each Node contains:

1. **int hash** (The calculated hash)
2. **K key** (The key object)
3. **V value** (The value object)
4. **Node next** (Reference to the next node in case of a collision)

2. How put(K, V) Works

1. **Hashing:** Java calls `key.hashCode()` to generate an integer.
2. **Index Calculation:** The hash is converted into an index using the formula: $\text{index} = \text{hash} \& (n-1)$ (where n is the array size).
3. **Collision Handling:** If two different keys result in the same index, it is called a **Hash Collision**.
 - **Linked List:** Initially, nodes are stored as a Linked List at that index.
 - **Treeification (Java 8+):** If a single bucket exceeds **8 nodes**, the Linked List is converted into a **Red-Black Tree** for faster searching ($O(\log n)$).

3. HashSet Internal

A `HashSet` internally uses a `HashMap` to store its elements. When you add a value to a `HashSet`, it is actually stored as a **Key** in a `HashMap`, and a dummy constant object (called `PRESENT`) is used as the **Value**.

#Object Cloning in Java

To use the `clone()` method of the `Object` class, a class must follow these rules:

1. **Implement Cloneable:** A marker interface that grants permission to clone.
2. **Override clone():** Change the access modifier from `protected` to `public`.
3. **Handle Exception:** It throws `CloneNotSupportedException`.

Shallow Copy vs. Deep Copy:

- **Shallow Copy:** The default `super.clone()` copies primitive fields but only copies references for objects (both original and clone point to the same internal object).
- **Deep Copy:** You must manually create new instances for internal objects inside the `clone()` method.

#Interview Questions and Answers

Q1. Why should we override `hashCode()` when we override `equals()`?

Answer: According to the Java Contract, if two objects are equal according to `equals()`, they must have the same `hashCode()`. If you only override `equals()`, two "equal" objects will have different hash codes, causing them to be stored in different buckets in a `HashMap`, making retrieval impossible.

Q2. What is the difference between `==` and `.equals()`?

Answer: `==` is an operator that compares memory addresses (references). The `.equals()` is a method in the `Object` class which also compares addresses by default, but it is intended to be overridden by classes (like `String` or `Integer`) to compare the actual **content** or state of the objects.

Q3. Can we override the `getClass()` method?

Answer: No. The `getClass()` method is declared as **final** in the `Object` class to prevent developers from changing the core logic of identifying an object's type at runtime.

Q4. What happens if two different keys have the same `hashCode`?

Answer: This is a **Hash Collision**. Both keys will be stored in the same bucket. In Java 8 and above, they are initially stored in a Linked List; if the bucket becomes too crowded (more than 8 nodes), the list converts into a Red-Black Tree to maintain $O(\log n)$ performance.

Q5. What is a "Marker Interface" in the context of the `Object` class?

Answer: A Marker Interface (like `Cloneable` or `Serializable`) has no methods. It simply serves as a signal or "tag" to the JVM. For example, the `clone()` method checks if the class implements `Cloneable`; if not, it throws an exception.

Q6. Why are `wait()`, `notify()`, and `notifyAll()` in the `Object` class instead of the `Thread` class?

Answer: Because synchronization (locking) happens at the **Object level**, not the thread level. Every object has a "monitor" or lock, and these methods allow threads to communicate based on the availability of that specific object's lock.